PRÁCTICA 3: SIMULACIÓN MCMC con Nimble

Introducción a la inferencia bayesiana basada en simulación

Nos planteamos de nuevo el modelo de regresión del banco de datos `cars` que trabajamos durante las dos prácticas anteriores.

1. Formula y simula dicho modelo de regresión, ahora con `Nimble`, haciendo uso de la función `nimbleMCMC`, mediante el proceso de ejecución "paso a paso" y haciendo uso de la función `pNimble` para su simulación en paralelo. En este último caso valora el beneficio de la simulación de tipo hamiltoniano en términos de su convergencia.

```
# Llamada mediante nimbleMCMC

library(nimble)
library(MCMCvis)

# Modelo de regresión lineal simple

modelCars = nimbleCode({
  for(i in 1:N){
    dist[i] ~ dnorm(beta0+beta1*velocidad[i], sd = sd.error)
  }
  beta0 ~ dflat()
  beta1 ~ dflat()
  sd.error ~ dhalfflat()
})
```

```
# Ejecución del modelo
data = list(dist = cars$dist)
constants = list(velocidad = cars$speed,N = length(cars$speed))
inits = function(){list(beta0 = rnorm(1,mean(cars$dist),10),
                        beta1 = rnorm(1,0,1),
                        sd.error = runif(1,0,10))}
param = c("beta0", "beta1", "sd.error")
resulCars = nimbleMCMC(code = modelCars, data = data, constants = constants,
                       inits = inits, monitors = param, nchains = 3, niter = 5000,
                       nburnin = 1000, thin = 4, summary = TRUE)
```

```
## |-------------|-------------|-------------|-------------|
## |-------------------------------------------------------|
## |-------------|-------------|-------------|-------------|
## |-------------------------------------------------------|
## |-------------|-------------|-------------|-------------|
## |-------------------------------------------------------|
```

```
# Monitorización de las cadenas simuladas
resulCars$summary
```

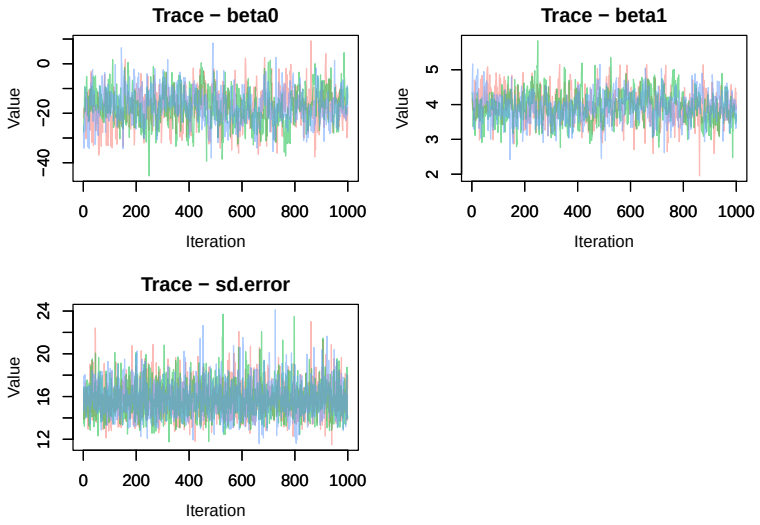
```
## $chain1
##                Mean     Median   St.Dev.  95%CI_low 95%CI_upp
## beta0    -18.037534 -17.943658 7.3959985 -32.550004 -2.958440
## beta1      3.967277   3.961886 0.4559204   3.069959  4.850363
## sd.error  15.767685  15.641550 1.6375732  13.077060 19.228377
##
## $chain2
##                Mean     Median   St.Dev. 95%CI_low 95%CI_upp
## beta0    -17.451888 -17.479202 6.9919147 -31.17836 -3.968470
```

```
## beta1      3.932101   3.919536 0.4370652   3.10661  4.767391
## sd.error  15.765950  15.581577 1.6450460  13.11587 19.213976
##
## $chain3
##                Mean     Median  St.Dev.  95%CI_low 95%CI_upp
## beta0    -17.112804 -17.104134 6.593347 -30.316174 -3.958878
## beta1      3.903649   3.905307 0.405590   3.115847  4.693952
## sd.error  15.758840  15.669533 1.611885  13.028548 19.309435
##
## $all.chains
##                Mean     Median   St.Dev.  95%CI_low 95%CI_upp
## beta0    -17.534075 -17.446248 7.0095092 -31.551673 -3.580199
## beta1      3.934342   3.928554 0.4339928   3.097375  4.789755
## sd.error  15.764158  15.632517 1.6310238  13.064799 19.306064
```

```
MCMCsummary(resulCars$samples)
```

```
##                mean        sd       2.5%        50%     97.5% Rhat n.eff
## beta0    -17.534075 7.0095092 -31.551673 -17.446248 -3.580199 1.01   643
## beta1      3.934342 0.4339928   3.097375   3.928554  4.789755 1.01   646
## sd.error  15.764158 1.6310238  13.064799  15.632517 19.306064 1.00  3000
```

```
# Vemos como MCMC summary incorpora estadísticos de convergencia que el
# propio summary del objeto Nimble no incorpora, lo que le hace una opción
# más interesante.
MCMCtrace(resulCars$samples, pdf = FALSE, type = "trace")
```




```
# Llamada paso a paso
t1 = Sys.time()
nimModel = nimbleModel(code = modelCars, data = data, constants = constants,
                       inits = inits())
compileNimble(nimModel)
```

```
## Derived CmodelBaseClass created by buildModelInterface for model modelCars_MID_2
```

```
modBuilt = buildMCMC(nimModel)
```

```
## ===== Monitors =====
## thin = 1: beta0, beta1, sd.error
## ===== Samplers =====
## conjugate sampler (3)
##   - beta0
##   - beta1
##   - sd.error
```

```
modComp = compileNimble(modBuilt, project = nimModel)
t2 = Sys.time()
resulCars2 = runMCMC(modComp, nchains = 3, niter = 2000, thin = 1, nburnin = 1000,
                 summary = TRUE)
```

```
## |-------------|-------------|-------------|-------------|
## |-----------------------------------------------------|

## |-------------|-------------|-------------|-------------|
## |-----------------------------------------------------|

## |-------------|-------------|-------------|-------------|
## |-----------------------------------------------------|
```

```
t3 = Sys.time()
cat("Tiempo de compilación:",t2-t1,"\n")
```

```
## Tiempo de compilación: 53.69488
```

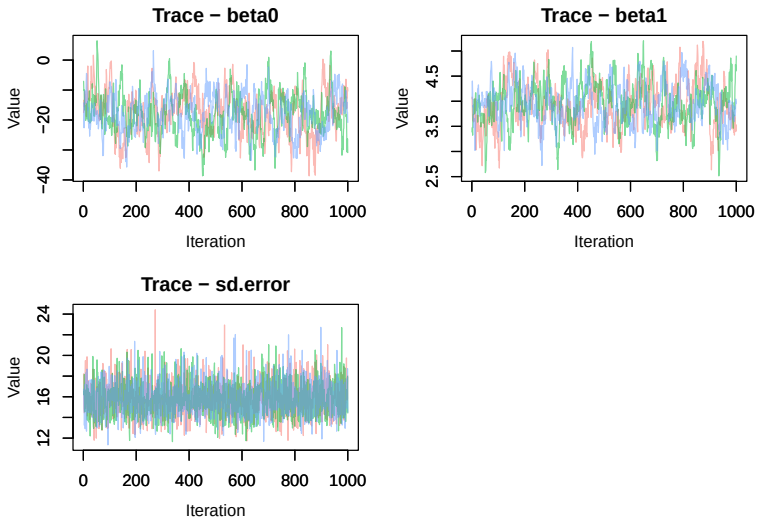
```
cat("Tiempo de simulación:",t3-t2,"\n")
```

```
## Tiempo de simulación: 0.7237451
```

```
# El tiempo de simulación es una parte ínfima del tiempo de ejecución total,
# por tanto Nimble muestra su ventaja computacional sólo cuando los tiempos
# de ejecución sean prolongados.


# Monitorización de las cadenas simuladas
MCMCsummary(resulCars2$samples)
```

```
##                mean        sd       2.5%        50%     97.5% Rhat n.eff
## beta0    -17.766521 6.8339044 -31.271868 -17.673300 -4.463687    1   166
## beta1      3.944178 0.4220962   3.117335   3.928633  4.774367    1   183
## sd.error  15.819528 1.6304802  12.964608  15.727487 19.379597    1  3000
```

```
MCMCtrace(resulCars2$samples, pdf = FALSE, type = "trace")
```




```
# La convergencia empeora ligeramente respecto a la llamada con nimbleMCMC,
# pero ahora tenemos control total del proceso lo que nos permite mejorar.
```

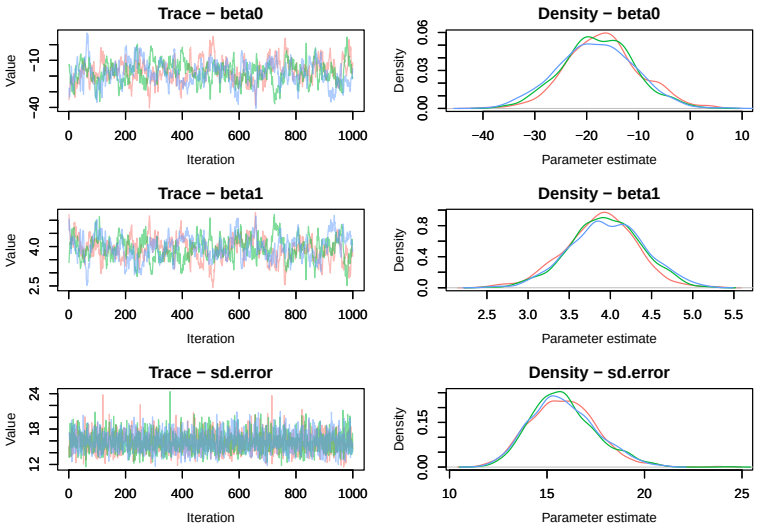
```
# Llamada mediante la función pNimble
source("https://raw.githubusercontent.com/MigueBeneito/pNimble/refs/heads/main/RutinasNi

resulCars3 = pNimble(code = modelCars, data = data, inits = inits,
                     monitors = param, constants = constants, nchains = 3,
                     thin = 1, niter = 2000, nburnin = 1000,
                     summary = TRUE, parallel = TRUE)
```

```
## Elapsed time: 62.66 seconds.
```

```
# Monitorización de las cadenas simuladas
MCMCsummary(resulCars3$samples)
```

```
##                mean        sd       2.5%        50%     97.5% Rhat n.eff
## beta0    -17.54651 7.1210230 -31.452510 -17.661810 -3.393171 1.05   167
## beta1      3.92776 0.4387385   3.049006   3.928455  4.772392 1.05   183
## sd.error  15.84411 1.6788862  12.935549  15.734227 19.399082 1.00  2891
```

```
MCMCtrace(resulCars3$samples, pdf = FALSE, ind = TRUE)
```



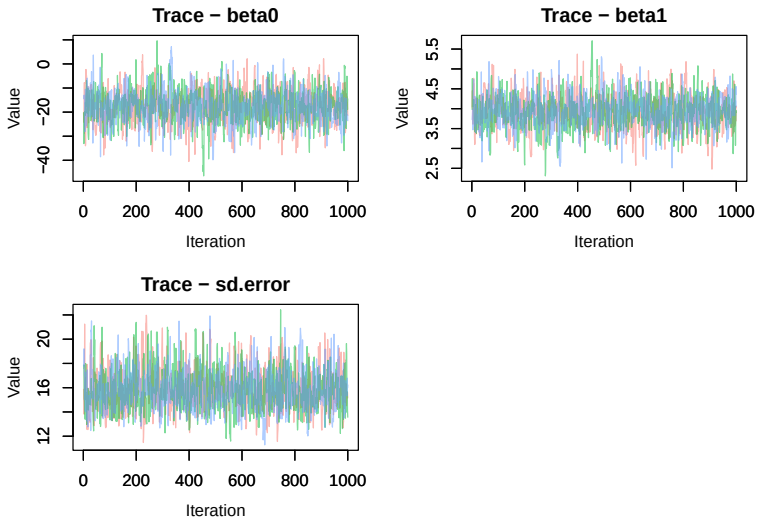

```
# Muestreo hamiltoniano
resulCars4 = pNimble(code = modelCars, data = data, inits = inits,
                     monitors = param, constants = constants, nchains = 3,
                     thin = 1, niter = 2000, nburnin = 1000,
                     summary = TRUE, HMC = TRUE, parallel = TRUE)
```

```
## Elapsed time: 196.28 seconds.
```

```
# Monitorización de las cadenas simuladas
MCMCsummary(resulCars4$samples)
```

```
##                mean        sd       2.5%        50%     97.5% Rhat n.eff
## beta0    -17.544073 7.0937666 -31.535264 -17.370170 -3.355938 1.01  1054
## beta1      3.923048 0.4335324   3.045066   3.923614  4.795468 1.01  1028
## sd.error  15.825558 1.7152985  12.857491  15.686550 19.735639 1.00  1118
```

```
MCMCtrace(resulCars4$samples, pdf = FALSE, type = "trace")
```




```
# La mejora en la convergencia con muestreo hamiltoniano es evidente
```

2. Repite el análisis sobre el efecto de insecticidas que llevaras a cabo en la práctica anterior, ahora en `Nimble` y empleando la(s) función(es) para su ejecución que te parezca más oportuna(s).

```
modelInsect = nimbleCode({
    for(i in 1:NInsectos){
      insectos[i] ~ dpois(efecto[i])
      # beta corresponde al afecto de cada insecticida y insecti asigna cada
      # observación al insecticida correspondiente
      log(efecto[i]) <- inter + beta[insecticida[i]]
    }
    inter ~ dflat()
    beta[1] <- 0
    for(j in 2:nInsecti){
      beta[j] ~ dflat()
    }
  })

# Ejecución de los modelos
```

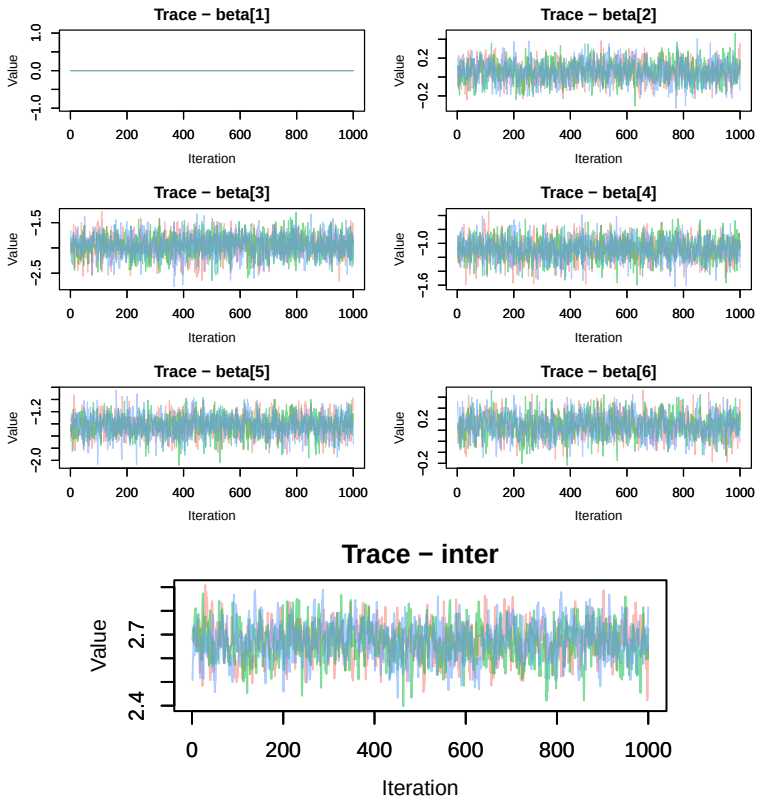
```
data = list(insectos = InsectSprays$count)
constants = list(nInsecti = 6, NInsectos = 72,
                 insecticida = as.numeric(InsectSprays$spray))
inits = function(){list(inter = rnorm(1), beta = c(NA, rnorm(5)))}
param = c("inter", "beta")
resulPois = pNimble(code = modelInsect, data = data, inits = inits,
                    monitors = param, constants = constants, nchains = 3,
                    thin = 1, niter = 2000, nburnin = 1000,
                    summary = TRUE, HMC = TRUE, parallel = TRUE)
```

```
## Elapsed time: 274.22 seconds.
```

```
# Monitorización de las cadenas simuladas
MCMCsummary(resulPois$samples)
```

```
##                mean         sd        2.5%        50%      97.5% Rhat n.eff
## beta[1]  0.00000000 0.00000000  0.00000000  0.0000000  0.0000000  NaN     0
## beta[2]  0.05817823 0.10873010 -0.16151938  0.0584160  0.2649341    1  1338
## beta[3] -1.95589672 0.22159842 -2.41788100 -1.9505343 -1.5329603    1  2451
## beta[4] -1.08296178 0.15604011 -1.38992748 -1.0818247 -0.7851594    1  2025
## beta[5] -1.42704167 0.17362987 -1.78153745 -1.4211135 -1.0990631    1  1721
## beta[6]  0.14088887 0.10454554 -0.06320881  0.1407984  0.3446635    1  1254
## inter    2.66938782 0.07706348  2.51613525  2.6708131  2.8172371    1   938
```

```
MCMCtrace(resulPois$samples, pdf = FALSE, type = "trace")
```



3. Repite el análisis del banco de datos `esoph` que llevaras a cabo en la práctica anterior, ahora en `Nimble` y empleando la(s) función(es) para su ejecución que te parezca más oportuna(s).

```
modelEsoph = nimbleCode({
    for(i in 1:Nncasos){
      ncasos[i] ~ dbin(p[i],ntot[i])
      logit(p[i]) <- inter + beta.edad[edad[i]] + beta.alc[alc[i]] +
        beta.tab[tab[i]]
    }
    inter ~ dflat()
    beta.edad[1] <- 0; beta.alc[1] <- 0; beta.tab[1] <- 0
    Pbeta.edad[1] <- 0; Pbeta.alc[1] <- 0; Pbeta.tab[1] <- 0
    for(j in 2:6){
      beta.edad[j] ~ dflat()
      Pbeta.edad[j] <- beta.edad[j] > 0
```

```
    }
    for(j in 2:4){
      beta.alc[j] ~ dflat()
      Pbeta.alc[j] <- beta.alc[j] > 0
    }
    for(j in 2:4){
      beta.tab[j] ~ dflat()
      Pbeta.tab[j] <- beta.tab[j] > 0
    }
})

data = list(ncasos = esoph$ncases)
constants = list(ncasos = esoph$ncases, ntot = esoph$ncases+esoph$ncontrols,
            edad = as.numeric(esoph$agegp), alc = as.numeric(esoph$alcgp),
            tab = as.numeric(esoph$tobgp), Nncasos = dim(esoph)[1] )
inits = function(){list(inter = rnorm(1), beta.edad = c(NA, rnorm(5)),
                        beta.alc = c(NA, rnorm(3)), beta.tab = c(NA, rnorm(3)))}
param = c("inter", "beta.edad", "Pbeta.edad", "beta.alc", "Pbeta.alc",
          "beta.tab", "Pbeta.tab")

resulLogis = pNimble(code = modelEsoph, data = data, inits = inits,
                     monitors = param, constants = constants, nchains = 3,
                     thin = 1, niter = 2000, nburnin = 1000,
                     summary = TRUE, HMC = TRUE, parallel = TRUE)
```

```
## Elapsed time: 186.95 seconds.
```

```
# Monitorización de las cadenas simuladas
round(MCMCsummary(resulLogis$samples),2)
```

```
##                mean   sd   2.5%   50% 97.5% Rhat n.eff
```

```
## Pbeta.alc[1]   0.00 0.00   0.00  0.00  0.00  NaN     0
## Pbeta.alc[2]   1.00 0.00   1.00  1.00  1.00  NaN     0
## Pbeta.alc[3]   1.00 0.00   1.00  1.00  1.00  NaN     0
## Pbeta.alc[4]   1.00 0.00   1.00  1.00  1.00  NaN     0
## Pbeta.edad[1]  0.00 0.00   0.00  0.00  0.00  NaN     0
## Pbeta.edad[2]  0.99 0.10   1.00  1.00  1.00 1.01  1606
## Pbeta.edad[3]  1.00 0.00   1.00  1.00  1.00  NaN     0
## Pbeta.edad[4]  1.00 0.00   1.00  1.00  1.00  NaN     0
## Pbeta.edad[5]  1.00 0.00   1.00  1.00  1.00  NaN     0
## Pbeta.edad[6]  1.00 0.00   1.00  1.00  1.00  NaN     0
## Pbeta.tab[1]   0.00 0.00   0.00  0.00  0.00  NaN     0
## Pbeta.tab[2]   0.97 0.16   0.00  1.00  1.00 1.00  2339
## Pbeta.tab[3]   0.97 0.16   0.00  1.00  1.00 1.00  2237
## Pbeta.tab[4]   1.00 0.00   1.00  1.00  1.00  NaN     0
## beta.alc[1]    0.00 0.00   0.00  0.00  0.00  NaN     0
## beta.alc[2]    1.47 0.25   0.98  1.47  1.97 1.00  2402
## beta.alc[3]    2.02 0.28   1.48  2.02  2.58 1.00  2693
## beta.alc[4]    3.69 0.39   2.95  3.68  4.49 1.01  2479
## beta.edad[1]   0.00 0.00   0.00  0.00  0.00  NaN     0
## beta.edad[2]   2.55 1.38   0.33  2.38  5.96 1.01   576
## beta.edad[3]   4.41 1.34   2.27  4.24  7.73 1.01   590
## beta.edad[4]   4.99 1.34   2.88  4.84  8.26 1.01   578
## beta.edad[5]   5.56 1.34   3.44  5.38  8.88 1.01   575
## beta.edad[6]   5.47 1.37   3.24  5.30  8.74 1.01   619
## beta.tab[1]    0.00 0.00   0.00  0.00  0.00  NaN     0
## beta.tab[2]    0.45 0.23   0.00  0.45  0.91 1.00  2991
## beta.tab[3]    0.52 0.27  -0.01  0.52  1.06 1.00  2790
## beta.tab[4]    1.67 0.35   0.98  1.68  2.35 1.00  2768
## inter         -7.60 1.35 -10.94 -7.43 -5.44 1.01   587
```

```
MCMCtrace(resulLogis$samples, pdf = FALSE, type = "trace", ISB =FALSE,
          params =c("beta.alc[2]", "beta.alc[3]", "beta.alc[4]",
                    "beta.edad[2]", "beta.edad[3]", "beta.edad[4]",
                    "beta.edad[5]", "beta.edad[6]", "beta.tab[2]",
                    "beta.tab[3]", "beta.tab[4]", "inter"))
```